

CSCE 614 COMPUTER ARCHITECTURE · MODULE 10 · LECTURE NOTES

GPUs as Throughput Machines

A different answer to the same memory problem.

MODULE OUTCOMES

- Contrast latency-oriented and throughput-oriented design.
- Explain SIMT execution and warp divergence.
- Explain memory coalescing and its importance.
- Explain occupancy and what limits it.
- Decide whether a workload suits a GPU.

1 Two answers to one problem

CPUs and GPUs face the same memory wall from Module 05 and respond in opposite ways. Understanding the two philosophies explains almost every difference between them.

	CPU — latency oriented	GPU — throughput oriented
Goal	Minimise the time for one thread to finish.	Maximise total work completed per unit time.
Die area	Mostly cache and control logic: branch predictors, schedulers, reorder buffers.	Mostly arithmetic units and register file.
Latency strategy	Avoid it (large caches) or hide it within one thread (out-of-order execution).	Tolerate it by switching to another of thousands of resident threads.
Threads	A handful per core.	Thousands per SM; tens of thousands per device.
Strong at	Anything, including irregular and branchy work.	Regular data-parallel work with high arithmetic intensity.

THE MECHANISM THAT MAKES IT WORK

When a GPU warp issues a memory request and stalls, the scheduler issues a different resident warp on the very next cycle. With 64 warps resident on a streaming multiprocessor, something is almost always ready.

This is only possible because switching is **free**: every resident thread's registers remain allocated, so there is no state to save or restore. That is what the enormous register file — 256 KB per SM is typical — is for.

The direct consequence is that a GPU requires *far more threads than it has cores* to perform well.

Launching a kernel with too little parallelism leaves the latency unhidden and is the most common reason GPU code underperforms.

2 SIMT

Threads are grouped into **warps** of 32 (NVIDIA terminology; AMD calls them waves and uses 32 or 64). Every thread in a warp executes the same instruction in the same cycle, on its own data.

You write ordinary scalar code describing what one thread does, and the hardware executes 32 copies in lockstep. This is SIMD with a programming model that hides the vector width — considerably more pleasant to write than the intrinsics of Module 09, at the cost of a performance model that is less obvious.

2.1 Divergence

If threads within a warp take different branches, the hardware executes *both* paths, masking off the lanes that should not be active in each. The warp's cost is the sum of the paths, not the maximum.

```
// Divergence within every warp: both paths run for all 32 lanes
if (threadIdx.x % 2 == 0) heavy_a();
else                      heavy_b();          // cost = a + b

// Divergence at warp granularity: each warp takes one path
if ((threadIdx.x / 32) % 2 == 0) heavy_a();
else                      heavy_b();        // cost = max(a, b)
```

DIVERGENCE AT WARP BOUNDARIES IS FREE

A branch on which all 32 threads of a warp agree costs nothing — only one path executes.

So the goal is not to eliminate branches but to arrange that *neighbouring threads agree*. Sorting work by branch outcome, or assigning work so that a warp handles a homogeneous group, routinely converts a 2× penalty into none.

The worst case — all 32 threads taking different paths through a 32-way switch — costs 32×. This is the hardware explanation for the shader branching guidance in CSCE 641 Module 07.

3 Memory coalescing

GPU memory is read in wide transactions — typically 128 bytes. If the 32 threads of a warp access 32 consecutive 4-byte elements, that is exactly one transaction, fully used.

```
// Coalesced: thread i reads element i
float v = data[blockIdx.x * blockDim.x + threadIdx.x];
// -> 32 consecutive floats = 128 bytes = one transaction

// Uncoalesced: thread i reads element 32i
float v = data[(blockIdx.x * blockDim.x + threadIdx.x) * 32];
// -> 32 separate transactions, 4 useful bytes out of 128 each
```

The uncoalesced version moves 32 times the memory traffic for the same data. This is Module 06's stride problem with a harsher penalty, and it is the single most important GPU optimisation: a kernel with uncoalesced access is frequently an order of magnitude below its potential, regardless of anything else.

The practical rule is the same as on the CPU, stated differently: arrange that consecutive thread indices touch consecutive addresses. This is yet another argument for struct-of-arrays layout, and it is why GPU code is written that way almost without exception.

3.1 The memory spaces

Space	Scope	Latency	Managed by
Registers	One thread	~0	Compiler. Use as many as occupancy allows.
Shared memory / LDS	A thread block	~20 cycles	You. A programmer-managed cache — this is how the blocking of Module 06 is done explicitly.
L1 / texture cache	One SM	~30 cycles	Hardware.
L2	The whole device	~200 cycles	Hardware.
Global memory (VRAM)	The whole device	400–600 cycles	You, through allocation and access pattern.

Shared memory is the distinctive one. It is a small, fast, explicitly managed scratchpad shared by a thread block, and using it well is essentially the blocking transformation of Module 06 performed by hand: load a tile cooperatively, synchronise, compute against the tile repeatedly, move on.

4 Occupancy

$$\text{occupancy} = \text{resident warps} / \text{maximum resident warps}$$

Occupancy determines how many warps are available to hide latency. It is limited by whichever resource is exhausted first:

- **Registers per thread.** A kernel using many registers allows fewer resident threads, since the register file is fixed.
- **Shared memory per block.** A block claiming a large allocation allows fewer blocks per SM.
- **Block size.** Blocks that are too small waste scheduling slots; typical good values are 128–256 threads.

HIGHER OCCUPANCY IS NOT AUTOMATICALLY BETTER

Occupancy is a means to latency hiding, not an end. A kernel with high arithmetic intensity and few memory accesses has little latency to hide, and may perform better with more registers per thread and lower occupancy.

The quantity to optimise is throughput. Occupancy is one input to it, and tuning occupancy without measuring throughput is a classic way to spend a day for nothing.

5 Is this workload a GPU workload?

Favours GPU	Favours CPU
Thousands of independent work items	Few threads, or heavy dependence between them
Regular, predictable memory access	Pointer chasing, trees, irregular graphs
High arithmetic intensity (many ops per byte)	Low intensity — transfer cost dominates
Little or warp-aligned divergence	Heavily data-dependent control flow
Data already resident on the device	Data must round-trip across PCIe for each call

ALWAYS COUNT THE TRANSFER

PCIe delivers roughly 16–32 GB/s. GPU memory delivers 500–1000 GB/s. Moving data to the device and back is therefore slow relative to everything the device does once the data is there.

A kernel that runs 50× faster than the CPU version is worthless if the transfer takes a hundred times the kernel's runtime. Many disappointing 'GPU-accelerated' library functions are exactly this.

The requirement is that arithmetic per transferred byte be high, or that the data live on the device across many operations.

This is precisely why real-time graphics suits GPUs so well: vertex buffers, textures, and render targets live in VRAM permanently and are used every frame. The transfer was paid once, at load time.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
NVIDIA CUDA C++ Programming Guide	The authoritative reference. Chapters on the execution model and memory coalescing are the ones that matter here.
OLCF CUDA Training Series (free video + exercises)	A structured free course with hands-on labs. The best guided introduction available.
Onur Mutlu — GPUs and SIMT lectures	The architectural view, with the latency-hiding argument developed properly.
CMU 15-418 / Stanford CS149 — GPU architecture lectures	Kayvon Fatahalian's treatment, which is unusually good at explaining <i>why</i> GPUs are shaped the way they are. Also the basis of CSCE 735.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 Write a CUDA (or OpenCL, or compute shader) kernel for vector addition and measure its bandwidth. Compare against the device's theoretical peak and account for the gap.
- 2 Write coalesced and uncoalesced versions of the same kernel. Measure both and confirm the ratio is close to the transaction-size argument.
- 3 Demonstrate warp divergence: write a kernel that branches on `threadIdx.x % 2` and one that branches on `threadIdx.x / 32`. Measure both.
- 4 Implement tiled matrix multiplication using shared memory. Compare against the naive version, and relate the structure to the CPU blocking of Module 06.
- 5 Vary registers per thread (via `__launch_bounds__` or just by adding live variables) and plot occupancy against achieved throughput. Find a case where lower occupancy is faster.
- 6 Measure PCIe transfer bandwidth and compute, for a kernel of your choice, the minimum arithmetic intensity at which GPU execution beats CPU execution including transfer.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 How does a CPU hide memory latency, and how does a GPU? Why can the GPU switch threads for free?
- 2 What is a warp, and what happens when threads in one diverge?
- 3 Why is divergence at warp granularity free?
- 4 What is memory coalescing, and what does uncoalesced access cost?
- 5 What is shared memory for, and which CPU optimisation does using it correspond to?

- 6 Why is higher occupancy not always better?
- 7 Give the condition under which moving a computation to a GPU is worth the transfer cost.